

Practical Work 2: RPC File Transfer

Le Quy Ton
23BI14425

December 15, 2025

1 Introduction

The objective of this practical work is to upgrade a TCP file transfer system to use Remote Procedure Calls (RPC). We utilize Python's `xmlrpc` library to implement a file transfer service where binary data is encoded in Base64 and transmitted in chunks.

2 Design of the RPC Service

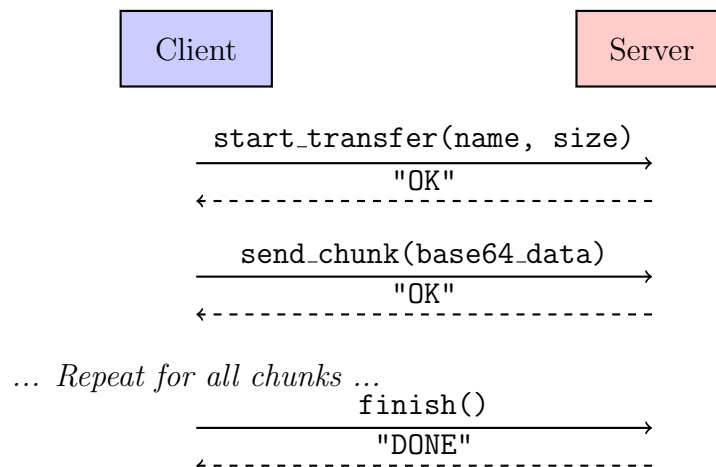
To handle file transfers effectively over XML-RPC (which is text-based), we designed the service with three specific remote procedures. This ensures that large files do not consume excessive memory and allows for basic session management.

The service exposes the following methods:

- **`start_transfer(filename, filesize)`**: Initializes the transfer on the server. The server checks if the file size is valid, resolves naming conflicts, and opens a file handle for writing.
- **`send_chunk(chunk_b64)`**: Receives a Base64-encoded string representing a part of the file. The server decodes it back to binary and writes it to the active file handle.
- **`finish()`**: Finalizes the transfer. The server closes the file handle and verifies if the received size matches the expected size.

2.1 Transfer Sequence Diagram

Below is the sequence of operations between the Client and the Server:



3 System Organization

The system is organized into a Client-Server architecture using the HTTP protocol as the transport layer for XML-RPC.

- **Client Side:** Reads the source file, encodes binary data to Base64 to ensure safe transmission over XML, and sends it sequentially.
- **Server Side:** Listens on port 8000. It manages a state machine (using a dictionary `_state`) to keep track of the current file being written, ensuring thread safety with locks.
- **Storage:** Files received by the server are stored in a dedicated directory named `received_files/`.

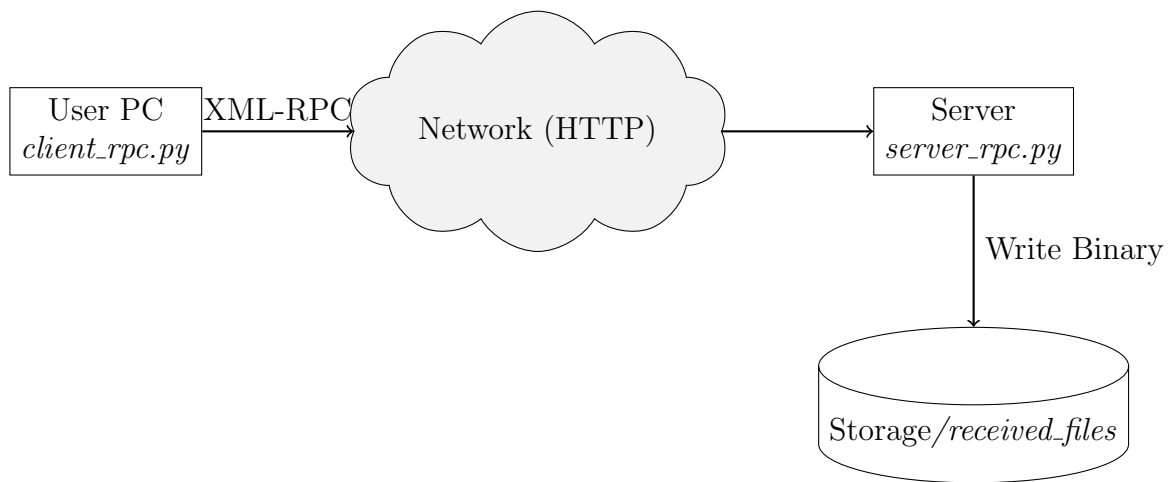


Figure 1: System Architecture

4 Implementation

4.1 Server Side Implementation

The server uses `SimpleXMLRPCServer`. Key logic for handling chunks is shown below:

```
1 def send_chunk(chunk_b64):
2     with _state_lock:
3         if not _state["active"]:
4             return "ERROR: no active transfer"
5         try:
6             # Decode Base64 back to binary
7             data = base64.b64decode(chunk_b64)
8             _state["file_obj"].write(data)
9             _state["received"] += len(data)
10        except Exception as e:
11            return f"ERROR: write failed: {e}"
12        return "OK"
```

Listing 1: Server Logic for Receiving Chunks

4.2 Client Side Implementation

The client reads the file in 64KB chunks, encodes them, and sends them via the proxy.

```
1 # Connect to server
2 proxy = xmlrpc.client.ServerProxy("http://localhost:8000/", allow_none=
    True)
3 proxy.start_transfer(basename, filesize)
4
5 CHUNK = 64 * 1024
6 with open(filename, "rb") as f:
7     while True:
8         chunk = f.read(CHUNK)
9         if not chunk:
10             break
11         # Encode binary chunk to Base64 string
12         b64 = base64.b64encode(chunk).decode()
13         proxy.send_chunk(b64)
14
15 result = proxy.finish()
16 print("Transfer complete:", result)
```

Listing 2: Client Loop for Sending File

5 Conclusion

In this practical work, I successfully transformed a raw TCP file transfer system into a Remote Procedure Call (RPC) based application. By utilizing Python's `xmlrpc` library, the complexity of socket management was abstracted, allowing the client to interact with the server through defined method calls.

Key achievements include:

- **Protocol Abstraction:** Moved from stream-based communication to a function-call model.
- **Binary Handling:** Successfully implemented Base64 encoding/decoding to transport binary files (images, executables) over the text-based XML protocol.
- **Chunked Transfer:** Implemented a chunk-based mechanism to handle large files without exhausting system memory.

While RPC introduces some overhead due to XML parsing and Base64 encoding compared to raw TCP, it significantly simplifies the development process and code readability.